

Основи роботи з фреймворком Spring Boot

Тема 1: Вступ

Бакалаврська сертифікатна програма
«Розробка бізнес-застосунків на Java»

Зміст лекції

1. Еволюція архітектури: Принципи IoC, DI та AOP у сучасній розробці.
2. Внутрішня механіка Spring Boot: Автоконфігурація та життєвий цикл об'єктів.
3. Аналіз ринку та аналогів: Місце Spring Framework у корпоративному сегменті.
4. Архітектурний стандарт: Модульні моноліти на базі Spring Modulith.
5. Продуктивність: Віртуальні потоки (Project Loom) та механізми Spring AOT.
6. Індустріальні стандарти: Синтаксис Java 25 та кращі практики.
7. Розгортання проєкту: Системи збірки та базовий REST-контролер.

1. Еволюція Java-архітектури та інверсія управління

Написання застосунку на чистій Java створює інфраструктурні проблеми:

- **Тісна зв'язність (Tight Coupling):** Прямі виклики `new Class()` прив'язують реалізації.
- **Бойлерплейт:** Багато дубльованого коду для керування ресурсами та транзакціями.
- **Складність тестування:** Важко підмінити залежності тестовими заглушками.

Spring пропонує **Hollywood Principle**: "*Don't call us, we'll call you*". Фреймворк сам керує вашим кодом.

Управління залежностями без IoC

Що станеться з проєктом на 100+ класів, якщо всі залежності створювати вручну через `new` у кожному сервісі?

Як це вплине на гнучкість коду, швидкість змінення бізнес-логіки та юніт-тестування?

Відповідь: Inversion of Control (IoC) & DI

Ручне створення через `new` створює жорстку залежність і ускладнює юніт-тестування.

Рішення Spring

- **IoC Container:** Сам створює об'єкти (Біни) та керує їхнім життєвим циклом.
- **Dependency Injection (DI):** Впроваджує необхідні залежності автоматично.

```
// Spring сам передасть репозиторій у сервіс через конструктор
public UserService(UserRepository repository) {
    this.repository = repository;
}
```

Ключові анотації (Стереотипи)

Для того, щоб Spring Container знайшов класи та перетворив їх на біни, використовуються стереотипні анотації:

- **@Component**: Базовий тип для будь-якого компонента.
- **@Service**: Позначає класи, що містять бізнес-логіку.
- **@Repository**: Позначає класи для доступу до бази даних (DAO).
- **@Controller** / **@RestController**: Відповідають за обробку вхідних HTTP-запитів.

Типи впровадження залежностей

Існує три основні способи передачі залежностей у клас:

1. **Constructor Injection:** Рекомендований підхід. Дозволяє робити поля `final` та гарантує, що об'єкт не буде створено без потрібних залежностей.
2. **Setter Injection:** Допускається виключно для опціональних залежностей.
3. **Field Injection** (`@Autowired` над полем): Вважається антипатерном, оскільки ускладнює написання модульних тестів без підняття контексту Spring.

Наскрізна логіка у багатьох методах

Як додати логування, перевірку прав доступу та відкриття транзакції у 100 різних методів без копіювання коду?

Які є варіанти окрім ручного виклику утилітарного методу на початку та в кінці кожного методу?

Відповідь: AOP та Декларативність

Ручне дублювання логіки у 100 методах порушує принцип **DRY** і ускладнює підтримку.

Рішення Spring: Aspect-Oriented Programming (AOP)

- Виносить наскрізну логіку в окремі **аспекти**.
- Створює динамічні проксі над бінами.
- Працює **декларативно** через анотації:

```
@Transactional // Spring сам відкриє і закриє транзакцію.  
public void processOrder(Order order) { ... }
```

2. Автоконфігурація та життєвий цикл компонентів

Spring Boot не генерує код, а застосовує умовні конфігурації.

Він перевіряє умови за допомогою набору анотацій `@Conditional` :

- **@ConditionalOnClass**: Конфігурація застосовується, якщо певний клас є у classpath.
- **@ConditionalOnMissingBean**: Застосовується, якщо розробник ще не створив бін певного типу вручну.

Анатомія @SpringBootApplication

Ця ключова анотація над головним класом є композицією трьох інших:

1. **@SpringBootConfiguration**: Позначає клас як джерело конфігурації (аналог `@Configuration`).
2. **@EnableAutoConfiguration**: Активує механізм розумного автоналаштування Spring Boot.
3. **@ComponentScan**: Вказує контейнеру сканувати поточний та всі вкладені пакети для пошуку `@Component` .

Приклад: Точка входу в застосунок

Ось як виглядає типовий головний клас Spring Boot додатку. Зверніть увагу на метод `main` та виклик `SpringApplication.run()` :

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

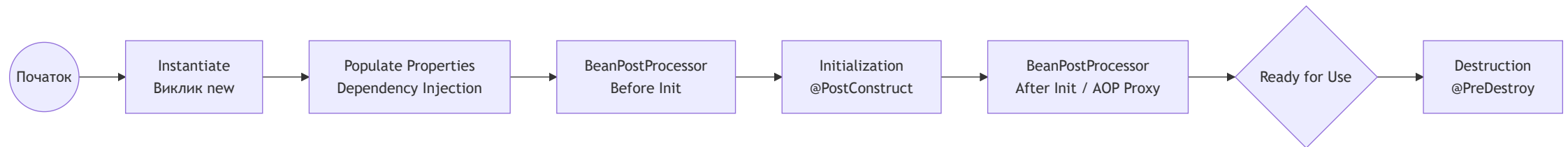
@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        // Запуск IoC-контейнера та автоконфігурації
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

Життєвий цикл Біна у Spring

1. **Instantiate**: Виклик конструктора.
2. **Populate**: Впровадження залежностей (DI).
3. **BeanPostProcessor (Before)**: Попередня обробка.
4. **Init**: Виклик `@PostConstruct` .
5. **BeanPostProcessor (After)**: Створення AOP-проксі.
6. **Ready / Destroy**: Робота та знищення (`@PreDestroy`).

Візуалізація: Життєвий цикл Біна



Області видимості (Bean Scopes)

Spring дозволяє гнучко контролювати кількість екземплярів біна за допомогою Scopes:

- **Singleton:** (За замовчуванням) Створюється один єдиний екземпляр на весь контекст застосунку.
- **Prototype:** Контейнер створює новий екземпляр щоразу, коли він запитується.
- **Request / Session:** Прив'язка екземпляра до тривалості конкретного веб-запиту або сесії користувача.

3. Аналіз ринку та місце Spring у корпоративній розробці

Spring — беззаперечний лідер розробки корпоративних Java-систем:

- **Багата екосистема:** Готові рішення для Web, Data, Security, Cloud, Batch, AI.
- **Стандарт індустрії:** Більше 70% ринку Java-розробки використовують Spring.
- **Зрілість та еволюція:** Підтримка новітніх можливостей Java 25, AOT та компіляції в нативний код.

Склад екосистеми Spring Portfolio

Фреймворк складається з десятків модулів, найпопулярнішими з яких є:

- **Spring Data:** Надає уніфікований підхід до роботи з реляційними (JPA) та NoSQL (MongoDB, Redis) базами даних.
- **Spring Security:** Промисловий стандарт для автентифікації, авторизації та захисту API (OAuth2, JWT).
- **Spring Cloud:** Набір інструментів для розробки складних розподілених мікросервісних систем.

Не лише Web-застосунки

Spring Framework традиційно асоціюється з REST API, проте його екосистема охоплює значно ширший спектр задач:

- **Spring Batch:** Обробка великих масивів даних (Big Data, ETL-процеси, нічні розрахунки).
- **Spring Integration:** Побудова систем обміну повідомленнями (Kafka) та IoT-рішень.
- **Spring AI:** Інтеграція сучасних нейромереж безпосередньо у бізнес-логіку.
- **Spring Shell:** Створення потужних консольних (CLI) утиліт для автоматизації.

Порівняння Java-фреймворків

Фреймворк	Переваги	Особливості
Spring Boot 4.1	Величезна екосистема, ком'юніті, підтримка AOT та Loom	Вище споживання пам'яті у класичному JVM-режимі
Jakarta EE	Офіційний стандарт Java	Повільніший розвиток, складніша конфігурація
Quarkus / Micronaut	Швидкий запуск під GraalVM із базової конфігурації	Менша екосистема, специфічний синтаксис

Вибір початкової архітектури проєкту

Ви починаєте новий проєкт. Що вибрати: одразу ділити застосунок на 10 мікросервісів чи писати один моноліт?

Які ризики має кожен із підходів на самому старті розробки?

Відповідь: Проблема передчасної мікросервісності

- Традиційний моноліт: Ризик отримати плутанину в коді (*Big Ball of Mud*).
- Мікросервіси на старті: Оверхед інфраструктури, мережеві затримки, складність розподілених транзакцій (Saga).

Передчасна мікросервісність — одна з найпоширеніших помилок архітектури.

4. Модульні моноліти на базі Spring Modulith

На курсі ми використовуємо **Spring Modulith** — модульний моноліт.

Переваги Spring Modulith

- **Єдиний пакет розгортання:** Простота розгортання як у моноліта.
- **Суворі межі модулів:** Автоматичні тести перевіряють заборонені зв'язки між пакетами.
- **Еволюційність:** Легко виділити модуль в окремий мікросервіс у майбутньому.

Як працює Spring Modulith?

Модулі спілкуються асинхронно через події, усуваючи прямі залежності:

```
// Модуль Order публікує подію
eventPublisher.publishEvent(new OrderCreatedEvent(orderId));

// Модуль Inventory слухає подію
@ApplicationModuleListener
public void on(OrderCreatedEvent event) { ... }
```

Цей підхід забезпечує слабку зв'язність (*Decoupling*) між доменами.

Інкапсуляція на рівні пакетів

Spring Modulith стимулює правильне використання модифікаторів доступу Java:

Публічним залишається лише API модуля (інтерфейси та класи подій). Внутрішні класи реалізації позначаються модифікатором доступу `package-private` (без вказання модифікатора). Це забороняє їх використання з інших модулів.

За дотриманням цих правил слідкують архітектурні юніт-тести під час збірки проєкту.

Навантаження на потік у Servlet API

Що станеться з сервером під час 100 000 паралельних HTTP-запитів при класичній моделі Servlet API (Thread-per-request)?

Чому платформні потоки ОС стають вузьким місцем?

Відповідь: Віртуальні потоки в Spring Boot 4.1

Платформні потоки ОС «важкі» (~1 МБ стеку). 100 000 потоків вичерпають пам'ять.

5. Високонавантажена багатопоточність (Project Loom)

- Легковажні потоки під управлінням JVM.
- Мільйони потоків без перевитрати ресурсів ОС.
- Простий синхронний код з продуктивністю асинхронного.

Налаштування Virtual Threads

Ввімкнення у Spring Boot 4.1 в 1 рядок в `application.properties` :

```
spring.threads.virtual.enabled=true
```

Механіка роботи

Під час блокуючого I/O (запит до БД чи мережі) віртуальний потік від'єднується (unmount) від потоку ОС, звільняючи платформовий потік для інших задач.

Обмеження віртуальних потоків

Незважаючи на потужність, Virtual Threads не є універсальним рішенням:

- **Pinning:** Використання ключового слова `synchronized` "прив'язує" віртуальний потік до потоку ОС, блокуючи його. Замість цього слід використовувати `ReentrantLock`.
- **CPU-bound задачі:** Для інтенсивних математичних обчислень та обробки даних віртуальні потоки не дають переваг, там краще використовувати класичні пули потоків.

Проблема тривалого запуску додатку

Чому традиційні Java-застосунки стартують кілька секунд або хвилин, що є критичним для Serverless (AWS Lambda)?

Що саме робить Spring під час запуску додатку?

Відповідь: Spring AOT Engine

Тривалий запуск викликаний скануванням `classpath` та рефлексією при старті JVM.

Компіляція Spring AOT (Ahead-of-Time)

- Аналізує Spring-контекст під час збірки проєкту.
- Генерує статичний код створення бінів.
- Усуває потребу в скануванні та рефлексії при запуску програми.

GraalVM Native Image & Closed-World Assumption

Компіляція Java-коду у бінарний нативний файл ОС:

- Час запуску: `< 50 мс` (ідеально для Serverless / Cloud Native).
- Споживання пам'яті: Зменшується в 3–5 разів.
- **Closed-World Assumption:** Усі класи мають бути відомі під час компіляції.

Складнощі нативної компіляції

Через правило Closed-World динамічні можливості Java (наприклад, рефлексія) у GraalVM працюють обмежено.

Для вирішення цієї проблеми Spring AOT автоматично генерує конфігураційні файли-підказки (Hints). Вони вказують компілятору GraalVM, які саме класи будуть використовуватися динамічно, щоб він міг включити їх у фінальний бінарний файл під час компіляції.

6. Сучасний стек Java 25 та індустріальні стандарти

Використання сучасного синтаксису Java 25:

```
// Record для DTO
public record UserDto(String name, int age) {}

// Pattern Matching y switch
return switch (payload) {
    case UserDto(var name, var age) when age >= 18 -> "Adult: " + name;
    default -> "Access denied";
};
```

Практика: Records як DTO у Spring

Контролери Spring Web ідеально працюють з `Record` для прийому (Deserialization) та повернення (Serialization) JSON. Це позбавляє від необхідності писати гетери, сетери та `equals` / `hashCode`.

```
public record CreateUserRequest(String username, String email) {}

@RestController
@RequestMapping("/users")
public class UserController {

    @PostMapping
    public String createUser(@RequestBody CreateUserRequest request) {
        // Spring автоматично перетворює JSON на Record
        return "User " + request.username() + " created with email " + request.email();
    }
}
```

Ізоляція доменних моделей (Sealed Classes)

Можливість обмеження ієрархії спадкування класів:

```
public sealed interface Payment permits CardPayment, CryptoPayment {}
```

Це дозволяє чітко зафіксувати набір підкласів. Компілятор гарантовано знає всі можливі реалізації `Payment`, що підвищує надійність при використанні конструкції `switch` (Pattern Matching).

Практика: Sealed Classes та Pattern Matching у Spring

Sealed Classes та Pattern Matching зручно використовувати для обробки різних результатів виконання бізнес-логіки та їх конвертації у HTTP-відповіді.

```
public sealed interface OrderResult permits Success, ValidationError {}
public record Success(String orderId) implements OrderResult {}
public record ValidationError(String reason) implements OrderResult {}

@PostMapping("/orders")
public ResponseEntity<?> createOrder() {
    OrderResult result = orderService.process();

    return switch (result) {
        case Success(var id) -> ResponseEntity.ok("Order: " + id);
        case ValidationError(var reason) -> ResponseEntity.badRequest().body(reason);
    };
}
```

Best Practices vs Anti-Patterns

Anti-Patterns

- Field Injection (`@Autowired` над полем): Важко тестувати.
- God Classes: Порушення Single Responsibility.
- Виклик `new` для Spring-бінів: Втрачається AOP та транзакції.

Best Practice

- Constructor Injection: Незмінність (`final` поля) та легке тестування.

7. Базовий REST-контролер (Створення API)

Мінімальний приклад для виконання лабораторної роботи:

```
@RestController
@RequestMapping("/api/hello")
public class HelloController {

    @GetMapping
    public StatusResponse sayHello() {
        return new StatusResponse("OK", "Spring Boot працює");
    }

    public record StatusResponse(String status, String message) {}
}
```

Семантика HTTP-методів

Створюючи REST API, необхідно використовувати правильні анотації для операцій:

- **@GetMapping:** Отримання даних (читання без модифікації).
- **@PostMapping:** Створення нового ресурсу.
- **@PutMapping:** Повне оновлення існуючого ресурсу.
- **@DeleteMapping:** Видалення ресурсу.
- **@PatchMapping:** Часткове оновлення ресурсу.

Залежності збірки для Spring Modulith

Для роботи з модульним монолітом використовують такі ключові залежності:

- **spring-boot-starter-web**: Забезпечує базову інфраструктуру REST API.
- **spring-modulith-starter-core**: Поставляє базові анотації та механізми перевірки меж модулів.
- **spring-modulith-events-api**: Надає інтерфейси для подійної взаємодії.
- **spring-modulith-starter-test**: Інструменти архітектурного тестування.

Задачі збірки у Gradle (Gradle Tasks)

У системі збірки Gradle використовують такі основні задачі:

- **compileJava**: Компілює сирцевий код проєкту під цільову версію Java.
- **test**: Виконує юніт-тести та верифікацію меж модулів.
- **bootRun**: Запускає Spring Boot застосунок у режимі розробки з вбудованим сервером.
- **bootJar**: Формує автономний виконаний JAR-архів зі всіма залежностями.
- **check**: Виконує комплексний аналіз коду та лінери.

Фази та команди збірки у Maven (Maven Goals)

У системі збірки Apache Maven життєвий цикл описується послідовними фазами:

- **mvn compile:** Компілює сирцевий код Java та копіює ресурси додатку.
- **mvn test:** Запускає автоматичне тестування та верифікацію архітектури Modulith.
- **mvn spring-boot:run:** Запускає додатковий екземпляр застосунку локально.
- **mvn package:** Збирає підсумковий виконаний JAR-архів у каталог `target`.

Формат Fat JAR (Executable JAR)

Після виконання команд збірки (`bootJar` або `mvn package`), Spring Boot створює так званий "Товстий JAR" (Fat JAR).

Цей архів відрізняється від звичайних тим, що містить:

1. Скомпільований код вашого застосунку.
2. Всі сторонні бібліотеки-залежності.
3. Вбудований веб-сервер (Tomcat, Jetty або Undertow).

Завдяки цьому застосунок запускається простою командою: `java -jar app.jar`.

Підсумки

Ключові висновки

1. Spring вирішує проблеми зв'язності за допомогою IoC та AOP.
2. **Spring Modulith** — базова архітектура проєктів на курсі.
3. Структура **Spring Boot** базується на конвенціях Maven/Gradle, де основний код (`src/main/java`), конфігурації (`src/main/resources/application.properties`) та тести мають чітко визначені місця, а анотація `@SpringBootApplication` є точкою входу.
4. **Virtual Threads** та **Spring AOT** забезпечують максимальну продуктивність.

Індивідуальне домашнє завдання (2 бали)

Дедлайн: до кінця наступної доби

Завдання: Створити Spring Boot проєкт і контролер зі своїм прізвищем, який повертатиме рядок, та інший контролер, який повертатиме параметри ПК.

(Результати обов'язково завантажити на distedu до дедлайну)

Як створити проєкт з нуля:

1. Перейдіть на start.spring.io (або використайте File -> New -> Project -> Spring Initializr в IntelliJ IDEA).
2. Оберіть систему збірки (Maven/Gradle) та Java (25).
3. Додайте залежність **Spring Web**.
4. Згенеруйте архів, розпакуйте та відкрийте в IDE.

Групове домашнє завдання (3 бали)

Дедлайн: 1 тиждень

Завдання: Зібрати команду. Визначитися з темою, узгодити ключові бізнес-кейси і затвердити з викладачем.

(Звітний документ обов'язково завантажити на distedu до дедлайну)

Критерії прийому та оцінювання:

- Вчасно зданий звітний документ-пропозиція.
- Кожен член команди відповідає за проєкт в цілому (немає жорсткого поділу ролей, всі працюють над усім).
- Тема достатньо складна для командної роботи (наявність кількох модулів або ролей користувачів).
- Бізнес-кейси логічні та покривають основний флоу застосунку.

Вимоги до звітнього документа (Групове ДЗ)

Звіт (у форматі Markdown або PDF) повинен містити:

1. **Назва проєкту та склад команди:** список учасників команди.
2. **Опис предметної області:** коротко про те, яку проблему вирішує ваш застосунок.
3. **Ключові бізнес-кейси (Use Cases):** 3-5 основних сценаріїв використання (наприклад, "користувач може додати товар у кошик та оплатити його").
4. **Базові сутності (Entities):** попередній перелік основних об'єктів (напр., `User`, `Order`, `Product`).
5. **Технологічний стек:** попередній план щодо БД (напр., PostgreSQL) та інших інструментів.